

Partiel Module d'ALGORITHMIQUE & SDA L3-Info

NOTES

- Les réponses aux questions doivent être claires et justifiées.
- Pour chaque algorithme (ou fonction en C), vous aurez soin d'expliquer les différentes actions de l'algorithme ainsi que le passage des paramètres.
- Chaque algorithme (ou fonction en C) supplémentaire utilisé doit être défini.

EXERCICE 1 (4.0 PTS)

- Soient un algorithme A et D(n) l'ensemble des jeux possibles de données de taille n. Notons par Coût(A(d)) la complexité en temps de l'algorithme A sur la donnée d ($d \in D(n)$).
 - Définir les complexités suivantes :
 - Complexité dans le meilleur des cas
 - Complexité dans le pire des cas
 - Complexité dans les moyennes des cas
 - Donner un exemple d'algorithme où les trois cas de complexités sont égaux (Justifier votre réponse).
 - Donner un exemple d'algorithme où les trois cas de complexités sont différents (Justifier votre réponse).
- Illustrer graphiquement le déroulement du tri fusion au tableau suivant : [3, 2, 6, 9, 1, 5, 8, 7, 0, 4].
- Illustrer graphiquement le déroulement du tri rapide au tableau suivant : [3, 2, 6, 9, 1, 5, 8, 7, 0, 4].

EXERCICE 2 (2.5 PTS)

- En ne comptabilisant que **les additions (lignes I)** ; trouver puis résoudre en fonction de n l'équation de la complexité de la fonction tata.

```

Fonction tata(n) → Entier
P.F n : Entier (E)
Debut
    Si n ≤ 1 alors
        Renvoyer n
    Sinon
        Renvoyer tata(n-1) + tata(n-2)      (I)
    Fsi
Fin
    
```

- En ne comptabilisant que **les multiplications (lignes I et II)** ; trouver puis résoudre en fonction de n l'équation de la complexité de la fonction toto.

```

Fonction toto(n, m) → Entier
Début
    Si n = 0 alors
        renvoyer 1
    Sinon
        Si estPair(n) alors
            renvoyer toto(n/2, m*m)      (I)
        Sinon
            renvoyer m*toto(n/2, m*m)    (II)
        FinSi
    FinSi
FIN
    
```

Paramètres Formels

n, m : entier. Paramètre en entrée

EXERCICE 3 (10.5PTS) : LISTES CHAÎNÉES

On souhaite gérer une **liste chaînée triée** par ordre croissant où les **données de la liste n'apparaissent qu'une seule fois**.

La structure de la liste chaînée est définie comme suit :

```

typedef int TElement ;
typedef struct cellule{
    TElement donnee;
    struct cellule * suivant;
}*Liste ;
    
```

- Ecrire en langage algorithmique ou en C, un algorithme qui recherche un élément donné dans une liste chaînée triée par ordre croissant donné. Cet algorithme retourne l'adresse de la cellule contenant l'élément s'il existe ou NULL si non.
- Ecrire en C, un algorithme qui insère un élément donné dans une liste chaînée triée donnée.
- Ecrire en C, un algorithme qui supprime un élément donné dans une liste chaînée triée donnée.

Par la suite, on souhaite gérer un tableau de listes chaînées qu'on nommé **table**. Les listes sont triées par ordre croissant. Pour cela on définit la structure en C comme suit :

```

typedef struct{
    int taille;      // la dimension du tableau
    Liste *tab;      // tableau de listes chaînées
}*Table ;
    
```

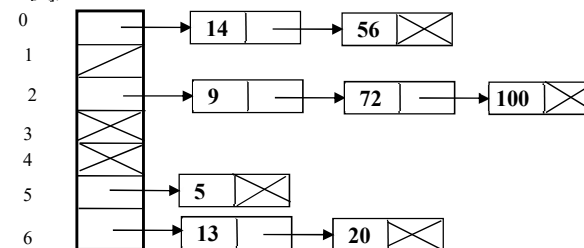
On suppose que la taille de la table est connue et les données seront insérées comme suit :

Pour insérer un élément « elt » dans la table on détermine d'une part sa position « pos » dans la table, et d'autre part son insertion dans la liste chaînée tab[pos]. **L'élément est inséré s'il n'existe pas dans la table (c'est-à-dire s'il n'appartient pas à la liste chaînée tab[pos]).**

La position est définie par la formule suivante : $pos = elt \text{ modulo } \text{taille}$.

Exemple : $taille=7$, les éléments 72, 14, 100, 13, 14, 56, 9, 5, 13, et 20 sont insérés dans la table comme ci-dessous :

72 modulo 7 = 2, 14 modulo 7 = 0, 100 modulo 7 = 2, 13 modulo 7 = 6,
56 modulo 7 = 2, 9 modulo 7 = 2, 5 modulo 7 = 5, 20 modulo 7 = 6,
72, 9, 100 sont insérés dans la liste chaînée tab[2], 14, 56 sont insérés dans la liste chaînée tab[0], etc



Remarque : Les données dans la table n'apparaissent qu'une seule fois ; par exemple 14 et 13.

- Ecrire en C, l'algorithme d'allocation mémoire d'une table en fonction d'une taille donnée.
- Ecrire en C, l'algorithme de libération mémoire d'une table donnée.
- Ecrire en langage algorithmique ou en C, l'algorithme d'initialisation d'une table donnée.
- Ecrire en C, l'algorithme d'insertion d'un élément donné dans une table donnée.
- Ecrire en langage algorithmique ou en C, un algorithme **itératif** qui remplit une table à partir d'un tableau donné composé de n éléments.
- Ecrire en langage algorithmique ou en C, un algorithme **récuratif** qui remplit une table à partir d'un tableau donné composé de n éléments.

10) Ecrire en langage algorithmique ou en C, un algorithme qui retourne le nombre d'éléments présents dans la table.

11) Ecrire en langage algorithmique ou en C ; un **algorithme itératif** qui déterminer l'élément de la table qui possède le plus d'éléments.

12) Ecrire en C ; un **algorithme récursif** qui déterminer l'élément de la table qui possède le plus d'éléments.

13) Ecrire en langage algorithmique ou en C, un algorithme qui affiche une table donnée.

14) Ecrire en C, l'algorithme de suppression d'un élément donné dans une table donnée.

15) Compléter en C le programme suivant :

```
void main(void){
TElement tab[DIM]={14, 72, 99, 7, 0, 13, 14, 5, 6, 2, 12, 15, 19,
33, 47, 13, 0}, n=17;
TElement elt;
Table t;

    // Allocation mémoire de la table
    ...
    // Initialisation de la table
    ...
    // Construction de la table à partir d'un tableau
        // appel de la version itérative
    ...
        // appel de la version récursive
    ...
    // Affichage du contenu de la table
    ...
    // Affichage du nombre d'éléments présent dans la table
    ...
    // Affichage de la plus grande liste de la table
        // appel de la version itérative
    ...
        // appel de la version récursive
    ...

    // suppression d'un élément de la table
    elt = 19;
    ...
    //Libération mémoire de la table
    ...
}
```

EXERCICE 4 (3.0 PTS) : LISTES CHAINÉES CIRCULAIRES AVEC SENTINELLE

Le but de cet exercice est de trier un tableau composé de n éléments (entiers par exemple), en utilisant une liste circulaire avec sentinelle comme structure intermédiaire. On suppose que la liste circulaire est donnée par l'adresse de la sentinelle.

- 1) Ecrire en C, l'algorithme d'insertion d'un élément donné dans une **liste triée** circulaire avec sentinelle donné.
- 2) En utilisant une liste circulaire avec sentinelle comme structure intermédiaire ; écrire en langage algorithmique ou en C, un algorithme de tri d'un tableau composé de n éléments donné.
- 3) Proposer en C un programme principal pour tester votre algorithme de tri.

*****Annexes*****

On suppose connu les primitives et les algorithmes suivants :

Structure Liste

Fonction donnee : Liste → TElement

Fonction suivant : Liste → Liste

Fonction initList : [] → Liste

Fonction estVideList : Liste → Booléenne

Fonction afficheList : Liste → [] // = procédure

Fonction longueurList : Liste → Entier

Structure Table

Fonction taille : Table → Entier

Fonction iEmeElt : Entier x Table → Liste